Ders 9 Genişletilmiş Notu: Yığın, İfade Değerlendirme ve Kuyruk Yapıları

Ders 9 Genişletilmiş Notu: Yığın, İfade Değerlendirme ve Kuyruk Yapıları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Erişim Şekline Göre Veri Yapıları
- 2. Yığın (Stack) ve LIFO Mantığı
- 3. Dizi ile Yığın Gerçekleştirme
- 4. Yığının Kullanıldığı Yerler
- 5. Aritmetik İfade Değerlendirme (Expression Evaluation)
- 6. Kuyruk (Queue) ve FIFO Mantığı
- 7. Dairesel Kuyruğun Gerçekleştirilmesi
- 8. Bağlı Liste ile Kuyruk
- 9. Yığındaki En Büyük Eleman Sorusu (Max Stack)
- Örnek Sorular
- Dersin Toparlaması

1. Erişim Şekline Göre Veri Yapıları

Hoca bugünkü konu olan yığın (stack) veri yapısına geçmeden önce, şimdiye kadar gördüğümüz iki yapının elemanlara nasıl eriştiğini hatırlattı.

Dizide (array) istediğimiz an istediğimiz göze doğrudan erişebiliriz. Örneğin `A[2] = 5` dediğimizde dizinin ikinci gözüne 5 değerini rahatlıkla yazarız. Bu, çoğu zaman ihtiyaç duyduğumuz bir erişim şeklidir: elemanları sıralarken, değerlerini değiştirirken, ekleme ya da silme yaparken, diziyi sıkıştırırken belli bir adrese doğrudan gitmek isteriz. Baştan veya sondan adım adım ilerlemeyiz; “ikinci göze 5 yazdım, dördüncü göze 9 yazdım” deriz.

Bağlı listede (linked list) ise, geçen hafta gördüğümüz gibi, elemanlara sırayla erişilir. Bir düğüm bir sonrakini gösterir. Ortadaki bir elemana doğrudan atlama imkânımız yoktur; baştan veya sondan adım adım ilerleyerek ona ulaşırız.

Yığın, erişim şekli bu ikisinden de farklı olan bir veri yapısıdır: Yığında yalnızca en son eklenen elemanla işlem yapılır.

2. Yığın (Stack) ve LIFO Mantığı

2.1 Günlük Hayattan Örnekler

Hoca yığının mantığını üç örnekle anlattı.

Undo ve redo. Hoca bir metin editörüne “kapı” yazdı, sonra undo yaptı. Editör harfleri son yazılandan başlayarak sırayla sildi: önce `ı`, sonra `p`, sonra `a`, en son `k`. Ardından redo yaptı; harfler yine sırayla geri geldi, ilk önce en son geri alınan. Undo, araya rastgele bir yerden girip örneğin `a`’yı silmez; her zaman en son yaptığımız işi geri alır. Ortadaki bir hatayı da

düzeltebiliriz elbette, ama undo ve redo'nun mantığı son yapılan işi geri almak ya da geri alınanı yeniden etkin hale getirmektir.

Tarayıcıda geri gitmek. Bir web tarayıcısında bir gazetenin sayfasına girdiniz, oradan "yazarlar" bağlantısına tıkladınız, oradan da bir yazarın yazısını açtınız. Geri (back) dediğinizde en son geldiğiniz yere, yani yazarlar sayfasına dönersiniz. Bir daha geri derseniz ondan önceki sayfaya dönersiniz. Her seferinde en son bulunduğunuz noktadan hareket edersiniz.

Fonksiyon çağırma. Bir programda `main` içinden `f1`'i çağırdınız, `f1` de `f2`'yi çağırıyor. `f2` bittiğinde, `f2`'yi çağırmadan önce `f1`'de kaldığınız yere dönmeniz gerekir. `f2`'den "pat diye" `main`'e dönmezsiniz. `f1` bittiğinde de `f1`'i çağırdığınız noktaya, yani `main`'de kaldığınız yere dönersiniz. Bu hiyerarşide de hep en son bulunduğunuz noktaya geri dönülür.

Bu örneklerin hepsinde yapılan şey, en son ne yapıldıysa onunla ilgili hareket etmektir. Bu yapıya **son giren ilk çıkar** (last in first out, LIFO) denir. Algoritmanız bu mantıkla çalışıyorsa kullanacağınız veri yapısı yığındır.

2.2 Yığının Görünümü: Push ve Pop

Bizim yazacağımız yığın aslında normal bir dizidir. Ama görsel olarak daha kolay anlaşılsın diye dikey çizilir. Hoca bunu bir bardağa su doldurmaya benzetti: İlk eleman bardağın dibine, yani 0. adrese yerleşir; sonrakiler yavaş yavaş yukarı doğru dolar.

Yığında iki temel işlem vardır:

- **push:** Yığına eleman ekleme.
- **pop:** Yığından eleman çekme.

Hocanın örneğinde en fazla 4 eleman alan bir yığına sırayla A, B, C konur. Sonra `x = pop` denince en üstteki C çıkar; bir daha `x = pop` denince B çıkar. Son gelen C, ilk çıkan olmuştur.

```
push A, push B, push C            x = pop   ->  x = C

3 |   |                           3 |   |
2 | C |  <- en üst                2 |   |
1 | B |                           1 | B |  <- en üst
0 | A |                           0 | A |
  +---+                             +---+
```

2.3 Donanımdaki Stack ile Farkı

Hoca, bilgisayarın donanımında da gerçekten bir stack bölgesi olduğunu, öğrencilerin bunu ilk dönem aldıkları assembly dersinde gördüğünü hatırlattı. Ancak bu derste yapılan şey, yüksek seviyeli (high level) bir yığın gerçekleştirmesidir. Gerçekteki stack bölgesini kullanmayacağız; kendi ihtiyacımıza göre bir yığın veri yapısını dizi kullanarak tasarlayacağız ve işlemlerimizi bu dizi üzerinden yapacağız.

Bunu neden yapıyoruz? Çünkü "yığına en son gelen ilk işlem görmeli" mantığıyla yaklaşılması gereken problemlerde, aradaki elemanlara rastgele erişmek tasarım gereği yapılmaması gereken bir şeydir. Yığın kullanarak yapının olması gerektiği gibi kullanılmasını sağlamış oluruz.

> **Hocanın vurgusu:** Yığında yalnızca en üstteki elemana erişilir. Aradaki elemanlara erişmek yasaktır; yapının anlamı budur.

2.4 Sınır Kontrolleri: isFull ve isEmpty

Yığın bir dizi olduğuna göre bir sınırı vardır. Eleman sayısını `MAX` olarak belirlediysek, yığının gözleri 0'dan `MAX - 1`'e kadar gider. Bu yüzden push ve pop dışında, hata oluşmasını önleyen iki fonksiyona daha ihtiyaç vardır:

- **isFull:** Yığın doluysa yeni eleman eklenmemelidir.
- **isEmpty:** Yığın boşsa eleman çekilmemelidir. Örneğin A, B, C'yi sırayla çektikten sonra bir daha pop yaparsak yığında eleman yoktur ve hata oluşur.

Akla şu soru gelebilir: Dizinin sınırı sorun oluyorsa bağlı liste kullanılamaz mı? Kullanılabilir, bunda bir sıkıntı yoktur. Ama yığında yaklaşık kaç eleman için yer gerektiğini biliyorsanız normal bir dizi yeterlidir.

3. Dizi ile Yığın Gerçekleştirme

3.1 Yapı Tanımı ve initialize

En fazla 100 elemanlık bir yığın düşünelim. Yapıda elemanları tutan bir dizi ve yığının en üstünü gösteren bir `top` indisi bulunur. `top`, push edildiğinde yeni elemanın yazılacağı gözü gösterir.

Initialize fonksiyonları her zaman başlangıç durumunu, yani değerlerin ilk halini gösterir. Bu derste `top` başlangıçta 0'dır. Hoca başka yolların da olduğunu söyledi: `top`'u -1'den başlatıp önce artırıp sonra yazmak da mümkündür, önce yazıp sonra artırmak da. Bu tamamen tercihe kalmıştır, aralarında ciddi bir fark yoktur.

```c
#include <stdio.h>
#define MAX 100

typedef struct {
    int items[MAX];
    int top;
} stack;

void initialize(stack *s)
{
    s->top = 0;
}
```

3.2 isEmpty

Yığını boşken `top = 0` olacak şekilde başlattık. O halde `s->top` 0'a eşitse yığın boştur ve fonksiyon 1 döndürür. Değilse yığında birtakım elemanlar vardır, `top` 0'ı göstermez ve fonksiyon 0 döndürür. Bu fonksiyonu pop kullanır.

```c
int isEmpty(stack *s)
{
    if (s->top == 0)
        return 1;
    else
        return 0;
}
```

3.3 isFull

Doluluk için `top`'un `MAX`'a eşit olup olmadığına bakılır. `MAX` 100 ise dizinin 0 ile 99 arasındaki gözleri kullanılır.

Hocanın örneği: `MAX = 3` olsun, gözler 0, 1, 2. Sırayla 5, 3 ve 7 push edilir. Her push'tan sonra `top` bir artar ve sonunda `top = 3` olur, yani `MAX`'ı gösterir. Ama 3 numaralı göz bize ait değildir; dizimizin yeri yalnız 0, 1, 2'dir. Demek ki `top == MAX` olduğunda yığın doludur ve fonksiyon 1 döndürür. Bu fonksiyonu push kullanır.

```c
int isFull(stack *s)
{
    if (s->top == MAX)
        return 1;
    else
        return 0;
}
```

> **Hocanın vurgusu:** Bu tip kontrolleri her zaman doğru yapmaya dikkat edin. Atlarsanız yanlış çalışan bir programınız olur.

3.4 push

Push fonksiyonuna iki şey aktarılır: yığına yerleştirilecek değer ve yığının adresi. Önce yığının dolu olup olmadığı kontrol edilir. Doluysa bir mesaj yazılıp 0 döndürülür. Hoca -1 de döndürülebileceğini, bunun tarzınıza kalmış olduğunu söyledi; önemli olan, ana programın push'un başarılı olup olmadığını ayırt edip ona göre davranabilmesidir.

Dolu değilse değer `items` dizisinin `top`'uncu gözüne yazılır, `top` bir artırılır ve başarıyla eklendiği için 1 döndürülür. Örneğin boş yığına A push edilince A 0. göze yazılır ve `top` artık 1'i gösterir.

```
int push(int value, stack *s)
{
    if (isFull(s)) {
        printf("stack is full");
        return 0;
    }
    s->items[s->top] = value;
    s->top++;
    return 1;
}
```

3.5 pop

Hoca pop'u iki türlü yazabileceğimizi söyledi:

- Değer bir pointer ile döndürülür; fonksiyonun dönüş değeri yalnızca pop işleminin başarılı olup olmadığını gösterir.
- Yığına yalnız pozitif değerler konacağı biliniyorsa, pop başarısızsa -1, başarılıysa değerin kendisi döndürülebilir.

Derste birinci yol yazıldı. Önce yığının boş olup olmadığına bakılır; boşsa işlem yapılamadığı için 0 döndürülür. Değilse önemli bir ayrıntı vardır: `top` şu anda bir sonraki boş gözü göstermektedir. Elemanın bulunduğu gözü göstermesi için önce `top` bir azaltılır (örneğin 1'den 0'a), sonra o gözdeki değer `value` değişkenine atanır. Başarıyla pop yapıldığı için 1 döndürülür.

```
int pop(int *value, stack *s)
{
    if (isEmpty(s))
        return 0;
    s->top--;
    *value = s->items[s->top];
    return 1;
}
```

Fonksiyonu çağıran yer, `value` değişkeninin adresini ve yığını gönderir. Pop işleminin sonucu bu `value` değişkeninin içine gelmiş olur.

3.6 peek

Yığın için bir fonksiyon daha vardır: peek. Peek pop'a benzer, tek farkı en üstteki elemanı yalnızca görmemizi sağlamasıdır. Elemanı yığından almaz, `top`'u değiştirmez. Örneğin yığında 0, 1, 2 numaralı gözlerde A, B, C varsa `top` 3'ü gösterir; peek size C'yi verir ve yığın aynen kalır. En üstteki eleman `items[top - 1]` gözündedir. Dönüş değeri, pop'taki gibi başarıyı göstermek için kullanılabilir.

Hoca peek'in kodunu öğrencilere bıraktı; çözüm "Örnek Sorular" bölümündedir.

3.7 Doğrulama

Yukarıdaki fonksiyonlar, peek eklenerek tek bir dosyada `gcc -Wall -Wextra` ile uyarısız derlendi. A, B, C push edildikten sonra peek C'yi verdi ve `top` 3'te kaldı; ardından üç pop sırayla C, B, A'yı verdi; boş yığından pop ve peek 0 döndürdü.

4. Yığının Kullanıldığı Yerler

Hoca, günlük kullandığımız sistemlerde yığın kullanılarak gerçekleştirilmiş algoritmaları şöyle sıraladı:

- Undo ve redo işlemleri.
- Fonksiyon çağırma (function call). Burada yığın kesinlikle kullanılır.
- Web sayfalarında ileri ve geri gitme.
- Aritmetik ifade değerlendirme (expression evaluation).

Hoca, öğrencilerin infix, postfix ve prefix gösterimlerini ayrık matematik (discrete mathematics) dersinde gördüklerini öğrendi. Öğrenciler yığınla çözümü de çizim olarak gördüklerini söyledi. Hoca yine de kısaca, çizim üzerinden göstereceğini; kodu göstermeyeceğini, çünkü kodun push, pop, isEmpty ve isFull kullanılarak rahatça yazılabileceğini söyledi.

> **Ek bilgi (derste anlatılmadı):** Infix gösterimde işlemci iki işlenenin arasına yazılır (`a + b`). Postfix gösterimde işlemci işlenenlerden sonra gelir (`a b +`), prefix gösterimde ise önce gelir (`+ a b`). Postfix ve prefix gösterimde parantez ve işlem önceliği kuralına gerek kalmaz; işlem sırası, ifadenin yazılış sırasından anlaşılır.

5. Aritmetik İfade Değerlendirme (Expression Evaluation)

5.1 Neden Yığın Gerekir?

Matematiksel ifadelerde işlem önceliği vardır. Hoca önce, toplamadan önce çarpmanın yapılması gereken kısa bir örnek gösterdi: Önce `2 * 4` yapılır, ama toplama unutulmamalıdır; çarpma bittikten sonra dönüp toplama, en son da çıkarma yapılır.

Sonra ifadeyi büyüttü:

$$7 + 5 \cdot 4 / 3 \cdot 2 - 8$$

Burada sıra şöyledir:

1. $5 \cdot 4$
2. sonucu 3'e bölmek
3. sonucu 2 ile çarpmak
4. sonucu 7 ile toplamak
5. en son 8'i çıkarmak

Hocanın deyişiyle "gittikçe evden uzaklaşıyoruz": 7 ile toplama işlemini baya bir süre aklımızda tutmamız, bu hiyerarşiyi bozmadan en sonunda kaldığımız yerden devam etmemiz gerekir. Bu, yığına çok uygun bir yapıdır. Derleyici matematiksel ifadeleri çözerken yığın kullanır ve bu sayede hiyerarşiyi bozmadan, işlem önceliklerine göre işlemleri gerçekleştirir.

> **Ders sonrası not:** Hoca bu ifadenin yalnızca işlem sırasını verdi, sonucunu hesaplamadı. Sıra yukarıdaki gibi izlendiğinde gerçek sayılarla sonuç $\frac{37}{3} \approx 12{,}33$ olur. C'de tüm işlenenler `int` ise `20 / 3 = 6` olacağı için sonuç 11 çıkar (python ve C ile doğrulandı).

5.2 İşlem Öncelikleri

Push ve pop için iki ayrı öncelik sırası vardır. Burada yalnızca bütün dillerde karşımıza çıkan temel işlem işaretleri ele alındı. Shift, bit düzeyinde ve/veya gibi dile bağlı işlemciler katılmadı; mantık aynı olduğu için gerektiğinde onlara da uygulanabilir.

Durum	En yüksekten en düşüğe
Push önceliği (yığına girerken)	(> *, / > +, -
Pop önceliği (yığından çıkarken)	*, / > +, - > (

- Push sırasında en yüksek öncelikli işaret açık parantezdir. Açık parantez varsa önce onun içindekiler bitirilmelidir.
- Pop sırasında açık parantez en düşük öncelikli işarettir. Parantezin içindeki işler bitmeden yığından alınmaz.

Kurallar derste örneklerle şöyle uygulandı:

- Yeni gelen işlemci, yığının üstündeki işlemciyle karşılaştırılır. Yığındaki işlemciler ancak gelen işaretten daha öncelikli ya da eşit öncelikli ise dışarı çıkar ve işlemleri yapılır.
- Kapa parantez yığına konmaz. Kapa parantez gelince yığındaki açık paranteze kadar olan bütün işaretler sırayla çıkarılıp işlenir; sonra açık parantez yığından atılır.
- İfade bitince yığında ne kalmışsa sırayla çıkarılıp işlenir.
- İki işlenen çekildiğinde işlem her zaman **alttaki (işlem) üstteki** şeklinde yapılır.

5.3 Örnek 1: `x = 3 + 5 * 8 - 2` (İki Yığınla)

Hoca önce `x = 3 * 5 + 7` demeyi düşündü, sonra parantez karışmasın diye parantezsiz `3 + 5 * 8` örneğini seçti ve sonuna `- 2` ekledi. İki yığın tanımlanır: işlenen (operand) yığını ve işlemci (operator) yığını. Hoca sayılarla gösterdi, ama gerçekte bunlar `x = aa + bb * cc` gibi değişkenler olur.

Gelen	İşlenen yığını	İşlemci yığını	Açıklama
3	3		
+	3	+	
5	3, 5	+	Yapacak iş yok
*	3, 5	+, *	+, *'dan daha az öncelikli; yığında kalır
8	3, 5, 8	+, *	
-	3, 40	+	*, -'den öncelikli: İşlem 1, $5 \cdot 8 = 40$
	43	-	+ ile - eşit öncelikli: İşlem 2, $3 + 40 = 43$; sonra - yığına girer
2	43, 2	-	
;	41		İfade bitti: İşlem 3, $43 - 2 = 41$

x'e 41 yazılır. Dikkat edilecek nokta, iki yığın kullanılmasıdır: birinde değerler, diğerinde işlem işaretleri saklanır. İşlem 1'de yığından önce 8, sonra 5 çekilir; "alttaki işlem üstteki" kuralıyla $5 \cdot 8$ yapılır.

5.4 Örnek 2: `3 * (5 + 8 / 2) - 4` (İki Yığınla)

Aradan sonra hoca parantezli bir örnek çözdü.

Gelen	İşlenen yığını	İşlemci yığını	Açıklama
3	3		
*	3	*	
(	3	*, (	Push önceliğinde (en yüksek; * çıkamaz
5	3, 5	*, (	
+	3, 5	*, (, +	Pop önceliğinde (en düşük; çıkamaz
8	3, 5, 8	*, (, +	

Gelen	İşlenen yığını	İşlemci yığını	Açıklama
/	3, 5, 8	*, (, +, /	/ daha öncelikli; + çıkamaz
2	3, 5, 8, 2	*, (, +, /	
)	3, 5, 4	*, (, +	İşlem 1: $8/2 = 4$
	3, 9	*, (	İşlem 2: $5 + 4 = 9$
	3, 9	*	(yığından atılır
-	27	-	- daha düşük öncelikli: İşlem 3, $3 \cdot 9 = 27$
4	27, 4	-	
son	23		İşlem 4: $27 - 4 = 23$

Açık parantez çıkarılırken işlenen yığınına hiç bakılmaz; nasıl kapa parantez yığına yerleştirilmiyorsa, açık parantez de yalnızca işlemci yığınından pop edilir. Örneğin sonunda x'e 23 yazılır. Python ile yapılan iz, aynı dört işlemi ve 23 sonucunu verdi.

5.5 Infix İfadeyi Postfix'e Çevirme

Yukarıda yaptığımız, derleyicinin bir ifadeyi doğrudan çözme yöntemidir. Ama bizden yalnızca ifadenin postfix halini, örneğin başka bir yerde kullanmak için, isteselerdi ne yapardık? Bu da yığınla yapılır. Çünkü postfix'e çevirmek için de hangi işlemin hangi sırayla yapılacağını bilmek gerekir: Çarpma geldiğinde, ardından açık parantez geldiği için çarpmayı yapamayız; önce içerisi bitecek, sonra çarpma yapılacak.

Bu kez yalnızca işlemciler için bir yığın kullanılır. Gelen değişkenler veya sayılar sırayla doğrudan çıktıya yazılır.

`3 * (5 + 8 / 2) - 4` için:

Gelen	Postfix çıktı	İşlemci yığını
3	3	
*	3	*
(	3	*, (
5	3 5	*, (
+	3 5	*, (, +
8	3 5 8	*, (, +
/	3 5 8	*, (, +, /
2	3 5 8 2	*, (, +, /
)	3 5 8 2 / +	*
-	3 5 8 2 / + *	-
4	3 5 8 2 / + * 4	-
son	3 5 8 2 / + * 4 -	

Sonuç:

`3 5 8 2 / + * 4 -`

Bu ifadede dört işlem vardır: Önce $8/2$, sonucu 5 ile toplanır, bunun sonucu 3 ile çarpılır, en son sonuçtan 4 çıkarılır.

5.6 Postfix İfadeyi Çözme

Elimizde postfix ifade varken çözmek için bu kez yalnızca işlenenler (değişkenler veya sayılar) için bir yığın kullanılır. İşlenen gelince yığına konur. İşlemci gelince yığının üstündeki iki değer çekilir, işlem yapılır, sonuç yığına konur.

Gelen	Yığın	İşlem
3, 5, 8, 2	3, 5, 8, 2	
/	3, 5, 4	İşlem 1: $8/2 = 4$

Gelen	Yığın	İşlem
+	3, 9	İşlem 2: $5 + 4 = 9$
*	27	İşlem 3: $3 \cdot 9 = 27$
4	27, 4	
-	23	İşlem 4: $27 - 4 = 23$

> **Ders sonrası not:** Hoca postfix'e çevirirken önce 2'yi çıktıya yazmayı atladı, sonra fark edip ekledi. Postfix'i çözerken de bir an 3'ü yığında bırakıp çarpmayı yanlış yaptı; bunu da kendisi fark edip düzeltti ("3'ü geri stackte bırakmışım"). Yukarıdaki tablolar düzeltilmiş hâlidir ve python ile doğrulandı.

5.7 İki Yöntemin Karşılaştırılması

- **Tek seferde çözmek:** Infix ifade, biri değişkenler veya sayılar, diğeri işaretler için olmak üzere iki yığınla doğrudan çözülür.
- **İki aşamada çözmek:** Önce işaretler için bir yığın kullanılarak infix'ten postfix'e çevrilir. Sonra yalnızca değişkenler veya sayılar için bir yığın kullanılarak postfix ifade çözülür: işaret gelince yığının en üstündeki iki değer çıkarılır, işlem yapılır, sonuç yığına konur.

Hoca bu bölümün sonunda da soru gelmemesinden yakındı ve aklınıza takılan bir şey olursa sormanızı istedi.

6. Kuyruk (Queue) ve FIFO Mantığı

6.1 Kuyruk Nedir?

Yığının çok yakın akrabası kuyruktur. Yığında son giren ilk çıkarken, işleriniz **ilk giren ilk çıkar** (first in first out, FIFO) mantığıyla yürüyorsa kuyruk kullanılır.

> **Ders sonrası not:** Hoca kuyruğu tanıtırken "queue, yığın veri yapısı" dedi. Doğrusu: stack = yığın, queue = kuyruk.

Market ya da banka kuyruklarında olduğu gibi, hizmet önce ilk gelene verilir; sonra geliş sırasına göre diğerlerine. Hocanın örnekleri:

- **Klavye:** Bilgisayarınız çok yavaşken tuşlara basarsınız, örneğin 3, 5, 4. O an hiçbirini göremezsiniz, ama sonra tuşlar tam bastığınız sırayla ekranda görünür.
- **Yazıcı:** Ağdaki ortak bir yazıcıya gönderilen işler kuyruğa girer ve sırası gelenin işi yapılır. Bunun için ağa da gerek yoktur: Kendi yazıcınıza arka arkaya birkaç iş gönderirseniz, birincisi basılıp bitmeden ikincisi başlamaz.
- **İşletim sistemi:** CPU zamanlaması (CPU scheduling) ve giriş/çıkış (I/O) işlemleri.

Yalnızca sıranın başındakine erişmeniz gerekiyorsa, ortadaki bir elemana erişmek istemiyorsanız, hatta erişmemeniz gerekiyorsa, kuyruk veri yapısını kullanırsınız.

Kuyrukta iki temel işlem vardır:

- **enqueue:** Kuyruğun sonuna yeni eleman ekleme (insert).
- **dequeue:** Sırası gelen, yani kuyruğun başındaki elemanı çıkarma (remove).

6.2 Basit Kuyruk

Kuyruk da dizi ile gerçekleştirilir. En fazla 5 elemanlık bir kuyruk düşünelim. `front` kuyruğun başını, `rear` sonunu gösterir. Bu en temel yapıda kuyruğun başı hep 0. gözdür. Kuyruğa 3, 5, 7, 8 yerleştirilmiş olsun; `rear`, en son gelen 8'i gösterir.

```
MAX = 5
indis:  0   1   2   3   4
       [3] [5] [7] [8] [ ]
     front          rear
```

- `enqueue(9)`: 9 dördüncü göze yerleşir, `rear` onu gösterir.

- `x = dequeue()`: Kuyruğun başındaki 3'e hizmet edilir, `x = 3` olur. 3 dışarı çıktığına göre 5, 7, 8, 9 birer öne kaydırılır; artık 5 0. gözdedir. `front` bu yapıda her zaman 0'dır, `rear` 9'u gösterir.
- Bir daha `x = dequeue()`: `x = 5`, kalanlar yine kaydırılır ve kuyruk 7, 8, 9 olur.

Hoca bunu markette kuyruktaki insanlara benzetti: İşi biten kişi çıkınca herkes bir öne doğru yürür.

Bu yapının sıkıntısı karmaşıklıktadır. Enqueue'nun karmaşıklığı $O(1)$'dir, çünkü eleman yalnızca en sona yerleştirilir. Ama dequeue'nun karmaşıklığı $O(n)$'dir: Bir elemanı kuyruktan çıkarmak, arkasındaki bütün elemanları birer öne kaydırmak demektir. Örneğin kuyruk 5, 7, 8, 9, 2 iken 5'i çıkarmak, 7'yi en başa, 8'i arkasına, 9'u onun arkasına, 2'yi de onun arkasına taşımak demektir. Özellikle çok hareketli kuyruklarda bu istenen bir şey değildir.

6.3 Dairesel Kuyruk (Circular Queue)

Çözüm kuyruğu dairesel hale getirmektir. Kuyrukta 3, 5, 2, 1, 4 olsun. 3'ün işi bitince 3'ü çıkarıp bütün diziyi kaydırmak yerine `front`'u bir ilerletiriz. Yeni bir eleman geldiğinde, örneğin 9, dizinin sonuna gelinmişse başa dönülür ve 9 baştaki boş göze yerleşir; `rear` oraya geçer. Yalnızca kuyruğun başını ve sonunu takip edersek işlem yükü ortadan kalkar:

- Enqueue $O(1)$'dir: Gelen eleman kuyruğun sonuna yerleştirilir.
- Dequeue $O(1)$'dir: Eleman çıkarıldığında yalnızca `front` değiştirilir.

Hoca bir kez daha gösterdi. Durum 9, 5, 2, 1, 4; `front` 5'i, `rear` 9'u gösteriyor. `x = dequeue()` denince `x = 5` olur ve `front` 2'yi gösterir. Fiziksel olarak bir şey yapılmaz, 5 orada durur; ama `front` 2'yi, `rear` 9'u gösterdiği için 5'in gözü artık kullanılabilir. Ardından `enqueue(7)` denince 7 o göze, 5'in üzerine yazılır ve `rear` orayı gösterir.

```
dequeue öncesi:   [9] [5] [2] [1] [4]      front -> 5, rear -> 9
x = dequeue():    [9] [5] [2] [1] [4]      x = 5, front -> 2, rear -> 9
enqueue(7):       [9] [7] [2] [1] [4]      front -> 2, rear -> 7
```

Hoca bu sırada bir an "enqueue" ile "dequeue"yu karıştırdı ve hemen düzeltti: Çıkarma dequeue, ekleme enqueue'dur.

7. Dairesel Kuyruğun Gerçekleştirilmesi

7.1 Yapı ve initialize

Yığında yalnız bir `top` vardı; burada kuyruğun başını ve sonunu gösteren `front` ve `rear` olmak üzere iki indis, bir de elemanları tutan dizi vardır. İşlemler enqueue ve dequeue'dur. Yığındaki gibi, kuyruk boşken dequeue yapılmaması için isEmpty, doluyken eleman eklenmemesi için isFull gerekir. Başlangıçta `front` ve `rear`'ın ikisi de -1'dir.

```c
typedef struct {
    int items[MAX];
    int front, rear;
} queue;

void initialize(queue *q)
{
    q->front = -1;
    q->rear = -1;
}
```

7.2 isEmpty

Yalnızca `front`'un -1 olup olmadığına bakmak yeterlidir; `rear`'ı ayrıca kontrol etmeye gerek yoktur, çünkü kuyruk boşken yeni bir eleman gelince ikisi aynı anda değişir.

```c
int isEmpty(queue *q)
{
    if (q->front == -1)
```

```
        return 1;
    else
        return 0;
}
```

7.3 isFull

Dairesel kuyruğun dolu olduğu iki durum vardır:

- **Hiç eleman silinmemiştir.** Bu "çok masum" durumda `front` 0'ı, `rear` en son gözü gösterir ve bütün gözler doludur: `front == 0` ve `rear == MAX - 1`.
- **Arada eleman silinmiş ve kuyruk dönmüştür.** Hocanın örneğinde dizi `[5, 7, 2, 3]`, `rear` 7'yi (1. göz), `front` 2'yi (2. göz) gösterir. Kuyruk 2'den başlamış, 3'ten sonra başa dönmüştür; işlem sırası 2, 3, 5, 7'dir. Bütün elemanlar etkindir, yeni eleman eklenemez. Bu durum `front == rear + 1` ile anlaşılır.

Hoca karşı örnek olarak, `front`'un `rear`'dan büyük olduğu ama aralarında eskiden kalma, artık boş sayılan bir gözün bulunduğu bir kuyruk gösterdi (elemanlar 3, 5, 7, 9). Burada `front` `rear`'dan bir fazla değildir, yerleştirilebilecek boş bir göz vardır; kuyruk dolu değildir. Demek ki bu iki kontrolü yapmak yeterlidir.

```
int isFull(queue *q)
{
    if ((q->front == 0 && q->rear == MAX - 1) || (q->front == q->rear + 1))
        return 1;
    else
        return 0;
}
```

> **Ders sonrası not:** İki koşul tek ifadeyle `(q->rear + 1) % MAX == q->front` olarak da yazılabilir. `MAX` 1'den 7'ye kadar ve tüm `front`, `rear` indis çiftleri için iki yazımın aynı sonucu verdiği python ile denetlendi.

7.4 enqueue

Önce isFull çalıştırılır. Kuyruk doluysa bir mesaj yazılıp 0 döndürülür; yeni eleman yerleştirilemez.

Değilse iki durum vardır. Eleman eklerken normalde `front` ile hiçbir işimiz yoktur. Tek istisna kuyruğun boş olduğu andır: Initialize'da `front`'u -1 yapmıştık; yalnızca bu ilk seferde, `front` -1 ise 0 yapılır.

Ardından `rear` her zaman `(rear + 1) % MAX` olur. Hocanın örneği: Dört gözlü bir dizide 4, 11, 3, 8 var; 0. göz eskiden kullanılmış ama artık boş, `rear` 3. gözü gösteriyor. isFull'dan başarıyla geçtiysek, gelen yeni eleman 0. göze yerleşmelidir. $(3 + 1) \bmod 4 = 0$ olduğu için öyle olur. Daire olduğu için başa dönmeyi bu mod işlemi sağlar. Sonra değer `items` dizisinin `rear`'ıncı gözüne yazılır ve başarı için 1 döndürülür.

```
int enqueue(queue *q, int value)
{
    if (isFull(q)) {
        printf("queue is full");
        return 0;
    }
    if (q->front == -1)              /* yalnız kuyruk boşken, bir kez */
        q->front = 0;
    q->rear = (q->rear + 1) % MAX;
    q->items[q->rear] = value;
    return 1;
}
```

7.5 dequeue (Öğrencilere Bırakıldı)

Hoca dequeue'yu öğrencilere bıraktı ve internetteki hazır çözümlere bakmadan yazmalarını istedi: Yığındaki push ve pop mantığı öğrenildi, burada da büyük bir fark yoktur. İpucu olarak şunu verdi: Pop'ta olduğu gibi dönüş değeri başarıyı (0 ya da 1) göstersin, çıkarılan değer parametreyle dönsün. Dequeue'da `front` gözündeki eleman `value`'ya alınır; örneğin `front` 1. gözdeki 5'i gösteriyorsa `value = 5` olur, dönüş değeri 1 olur ve `front` artık 2. gözdeki 7'yi gösterir. Böylece 1 numaralı göz serbest kalır ve baştaki elemana artık erişilmez. Kesinleştirilmiş çözüm "Örnek Sorular" bölümündedir.

8. Bağlı Liste ile Kuyruk

8.1 Fikir

İkinci aradan sonra hoca kuyruğu kullanmanın üç yolunu özetledi:

1. Basit kuyruk: Yeni eleman sona eklenir, işi biten çıkınca diğerleri öne kaydırılır.
2. Dairesel kuyruk: Kaydırma yükünden kurtulmak için, örneğin 3, 5, 7'de 3'ün işi bitince yalnızca kuyruğun başı değiştirilir.
3. Kuyruğu bağlı liste yapısıyla kullanmak.

Öndekine hizmet edilip o gidiyor, yeni gelen arkaya ekleniyorsa, bu bir bağlı listeyle kolayca yapılabilir. Kuyruk A, B, C olsun; `front` A'yı, `rear` C'yi göstersin. A'nın işi bitince `front` B'yi gösterir. D gelince kuyruğun sonuna eklenir ve `rear` onu gösterir. Yeni eleman geldiğinde yalnızca `rear` pointer'ı, öndeki elemana hizmet edildiğinde yalnızca `front` pointer'ı değişir.

```
front                       rear
  |                           |
  v                           v
 [A | ·] -> [B | ·] -> [C | NULL]
```

Bu yapıda kuyruğun dolması gibi bir durum yoktur. Tek hata durumu, yeni düğüm için bellekte yer ayrılamamasıdır; bu ciddi bir problem olduğu için hata verip programdan çıkılabilir.

8.2 Yapılar

Bağlı listedeki her eleman bir değer ve kuyrukta kendisinden sonra geleni gösteren `next` adresini tutar. Kuyruğun kendisi ise yalnızca başını ve sonunu gösteren iki pointer'dan oluşur.

```
typedef struct qnode {
    int value;
    struct qnode *next;
} qnode;

typedef struct {
    qnode *front, *rear;
} queue;
```

8.3 Kuyruğu Yaratmak

Kuyruk yaratıldığında iki pointer vardır ama kuyrukta kimse yoktur. Bu yüzden ikisi de `NULL` gösterir. Başlangıç durumu budur.

```
queue *createQueue(void)
{
    queue *q = (queue *) malloc(sizeof(queue));
    q->front = NULL;
    q->rear = NULL;
    return q;
}
```

8.4 Yeni Düğüm: newNode

Bu fonksiyon, gelen eleman için bellekte yeni bir yer ayırır ve elemanı oraya yerleştirir. `qnode` tipinde `temp` adlı bir pointer tanımlanır ve bellek ayrılır. Yer ayrılamadıysa (bellek tamamen dolmuş olabilir ya da başka bir sebep olabilir) istenen hata işlemi yapılır. Sonra `temp`'in değerine yeni değer yazılır; bu düğüm son eleman olacağı için kimseyi göstermez, `next`'i `NULL` yapılır; en son `temp` döndürülür. Örneğin 1100 numaralı adreste bir yer ayrılır, değeri 9, `next`'i `NULL` olan bir düğüm "uzay boşluğunda" durur.

```
qnode *newNode(int newValue)
{
    qnode *temp = (qnode *) malloc(sizeof(qnode));
    if (temp == NULL)
        exit(1);
    temp->value = newValue;
    temp->next = NULL;
    return temp;
}
```

8.5 enqueue

Enqueue yeni gelen elemanı kuyruğa ekler. Önce newNode ile `temp` alınır. İki durum vardır:

- **Kuyruk boşsa** (`front == NULL`): Hem `front` hem `rear` yeni düğümü gösterir. Örneğin yeni düğüm 1500'deyse, kuyruğun başı da sonu da 1500'dür.
- **Kuyruk boş değilse:** Şu anki son düğümün `next`'i yeni düğümü gösterir, sonra `rear` yeni düğüme geçer. `front`'ta bir değişiklik olmaz.

Hocanın örneği: Kuyrukta 1500'de 9, 1700'de 8, 1800'de 17 var; `front` 1500'ü, `rear` 1800'ü gösteriyor. 1100 adresinde yeni bir eleman geldi. Önce `rear->next`, yani 1800'ün `next`'i 1100'ü gösterir; sonra `rear` 1800 değil 1100 olur. Böylece hem eski son eleman yeni elemanı göstermiş, hem de `rear` yeni elemana geçmiş olur.

Hoca bunu bir adım daha sürdürdü: Bu kez 1200 adresindeki 8 değerli bir düğüm gelir. `front` 1500, `rear` 1100'dür. Kuyruk boş olmadığından yalnız `rear` değişir: 1100'ün `next`'ine 1200 yazılır ve `rear` 1200 olur.

```
void enqueue(queue *q, int value)
{
    qnode *temp = newNode(value);
    if (q->front == NULL) {          /* kuyruk boş */
        q->front = temp;
        q->rear = temp;
    }
    else {
        q->rear->next = temp;
        q->rear = temp;
    }
}
```

Ana programda `queue *` tipinde bir değişken tanımlanır, önce kuyruk yaratılır, sonra örneğin `enqueue(q, 5)` gibi çağrılarla eleman eklenir. Enqueue'nun dönüş tipi `void`'dir.

Soru: Fonksiyonun parametresi `qnode` olmayacak mıydı? Daha önce stack tipini öyle tanımlamıştık. **Cevap:** Hayır. `qnode` yeni gelen düğümün tipidir. Kuyruğun kendisi `front` ve `rear` pointer'larını tutan `queue` tipidir. Bizim için önemli olan düğümün kendisi değil, `front` ve `rear`'ın nereyi gösterdiğidir; yeni düğümü de nasılsa birileri gösterecektir. Öğrenci, üst tarafta yazılan `qnode` adının kafasını karıştırdığını söyledi; newNode fonksiyonu yazılınca ayrım netleşti.

8.6 dequeue

Hoca, zor olduğu için bu dequeue'yu kendisi yazdı. Dequeue kuyruğun başındaki elemanı çıkarır.

> **Ders sonrası not:** Hoca bir yerde "çıkacak eleman kuyruğun en sonundaki eleman" dedi, ama hemen ardından kodu kuyruğun başındaki (`front`) elemanı silecek biçimde anlattı ("silmem gereken eleman şu eleman", front). Doğrusu: dequeue her zaman kuyruğun başındaki elemanı çıkarır.

Örnek kuyruk: 1000'de 8, 1100'de 9, 1200'de 5. `front` 1000'i, `rear` 1200'ü gösteriyor; 5'in `next`'i `NULL`.

1. `qnode` tipinde geçici bir `temp` değişkeni tanımlanır.
2. Önce kuyruğun boş olup olmadığı kontrol edilir. Hoca isEmpty'yi ayrıca yazmadı, öğrencilere bıraktı; kodda `front == NULL` kontrolünü elle yaptı. Boşsa "queue is empty" yazılır ve bir şey yapılmaz.
3. `value`, kuyruğun başındaki düğümün değeri olur: 1000'deki 8.
4. `front`, `front->next` olur: artık 1100.
5. 1000 numaralı düğümü kimse göstermez; `temp` hâlâ onu gösterdiği için `free(temp)` ile silinir.
6. İş bununla bitmez. Kuyrukta yalnız bir eleman varsa, örneğin 1000'de tek başına 8 varsa, onu silince `front` `NULL` olur ama `rear` hâlâ 1000'i gösterir. Bu yüzden `front == NULL` ise `rear` da `NULL` yapılmalıdır; çünkü kuyrukta kimse kalmamıştır.
7. Sonunda `value` döndürülür.

```
int dequeue(queue *q)
{
    qnode *temp;
    int value;
    if (q->front == NULL) {
        printf("queue is empty");
        return -1;
    }
    temp = q->front;
    value = temp->value;
    q->front = q->front->next;
    free(temp);
    if (q->front == NULL)            /* kuyruk boşaldı */
        q->rear = NULL;
    return value;
}
```

Yığında olduğu gibi burada da `value` parametre olarak aktarılıp dönüş değeri yalnızca çıkarmanın başarılı olup olmadığını gösterebilir. Hocaya göre her iki yol da olur.

Bu bölümdeki kod, isEmpty eklenerek `gcc -Wall -Wextra` ile uyarısız derlendi. 9, 8, 17, 5 eklenip boşalana kadar çıkarıldığında aynı sırayla 9, 8, 17, 5 geldi; son elemandan sonra `rear` `NULL` oldu ve boş kuyruktan dequeue −1 döndürdü.

9. Yığındaki En Büyük Eleman Sorusu (Max Stack)

9.1 Soru

Dersin sonunda hoca bir soru sordu ve bir sonraki derste konuşulacağını söyledi. Cevabının internette çok bulunacağını, ama bakmadan çözmeye çalışılmasını istedi.

Bir yığında, herhangi bir anda yığındaki en büyük elemanı bilmemiz isteniyor. Örneğin 3 gelir, en büyük 3'tür; 5 gelir, 5 olur; 2 gelir, hâlâ 5'tir; 7 gelir, 7 olur. Şimdi 7 yığından çıktı. En büyük kaç olmalı? 5. Bunu nasıl bulacağız? Yalnızca üç işlem yapabiliyoruz: push, pop ve peek. Aşağılara inip bakma şansımız yok.

Hoca soruyu bir kez daha anlattı: 3 gelir, en büyük 3; 1 gelir, en büyük 3; 7 gelir, en büyük 7; 4 gelir, hâlâ 7; 9 gelir, en büyük 9 olur. Şimdi 9'u pop ettik. En büyük eleman 7 olmalı. Bunu nasıl bulursunuz?

> **Ders sonrası not:** Hoca soruyu sorarken bir an "7 kuyruktan çıktı" dedi; soru yığınla ilgilidir, doğrusu "7 yığından çıktı" olmalıdır.

9.2 Önerilen Fikirler ve Neden Yetmedikleri

Sohbet kutusundan ve sesli olarak birkaç öneri geldi:

- **Pop edilenleri bir yere toplamak:** Hoca, pop ettiklerinin "uzay boşluğuna" gittiğini, onları da bir yere toplamaya çalışmanın işi çok karıştıracağını söyledi. Yine de yedek bir yer kullanılabileceğini belirtti.
- **Başka bir pointer ya da geçici elemanla aralara erişmek:** Olmaz. Yığında yalnızca en üste erişilebilir; aralara erişmek yasaktır.
- **Tek bir `max` değişkeni:** 9 çıkınca bir önceki en büyüğü bilemezsiniz.
- **İki değişken (`max` ve önceki `max`):** Hoca bunu bir örnekle çürüttü. 3, 1, 7, 2, 9, 5 sırayla push edilir; en büyük 3, 3, 7, 7, 9, 9 olur. Önce 5, sonra 9 pop edilir; en büyük 7 olmalıdır. Diyelim ki `max = 9` ve önceki `max = 7` tutuyordunuz, 7'yi bildiniz. Ama ardından 7 de pop edilirse, en büyüğün 3 olduğunu nasıl söyleyeceksiniz? Yalnızca iki eleman tutarsanız ondan da öncekini bilemezsiniz.

Hoca bu sırada sorulan soruları tam ekranda göremediğini, bu yüzden soruların yüksek sesle sorulmasını istedi.

9.3 Çözüm: Max Yığını

Bir öğrenci şunu önerdi: Yeni gelen en büyük değerleri başka bir yığına koyalım; silindikçe bir öncekini alalım. Örneğin 3, 7, 9 yazdık; 9 silinince 7'yi, 7 silinince 3'ü alırız. Hoca "olaya yaklaştın, doğrusu bu aslında" diyerek bunu onayladı ve adım adım gösterdi.

Asıl yığının yanında, en büyük değerleri tutan bir **max yığını** (max stack) tutulur:

- Push edilen değer o anki en büyükten büyükse max yığınına da push edilir.
- Pop edilen değer max yığınının üstündekine eşitse max yığınından da pop edilir.
- En büyük eleman her zaman max yığınının en üstündedir.

Hoca ayrıca elde bir `max` değişkeni de tuttu. Bunun zorunlu olmadığını, max yığınının en üstündeki elemanın zaten her zaman en büyük olduğunu söyledi; değişken, yığına sürekli peek ile erişmemek için kullanılır, çünkü peek de sonuçta bir işlemdir.

İşlem	Yığın	Max yığını	En büyük
push 3	3	3	3
push 1	3, 1	3	3
push 7	3, 1, 7	3, 7	7
push 5	3, 1, 7, 5	3, 7	7
push 9	3, 1, 7, 5, 9	3, 7, 9	9
push 11	3, 1, 7, 5, 9, 11	3, 7, 9, 11	11
push 2	3, 1, 7, 5, 9, 11, 2	3, 7, 9, 11	11
pop (2)	3, 1, 7, 5, 9, 11	3, 7, 9, 11	11
pop (11)	3, 1, 7, 5, 9	3, 7, 9	9
pop (9)	3, 1, 7, 5	3, 7	7
pop (5)	3, 1, 7	3, 7	7
pop (7)	3, 1	3	3

11 çıkarken max yığınının üstünde 11 olup olmadığı kontrol edilir; varsa oradan da çıkarılır ve en büyük 9 olur. 5 çıkarken max yığınıyla hiç ilgilenilmez, çünkü en büyük 7'dir ve 5 ile ilgisi yoktur. 7 çıkınca en büyük 3 olur; o an asıl yığının üstünde 1 vardır.

Hoca, öğrencilerin düşünme sırasını da beğendi: Önce bir değişken yeter mi diye bakıldı; iki değişken de kurtarmadı; demek ki bir dizi tutmak gerekiyordu. Yalnızca o anki en büyük elemanları sırayla saklayan yedek bir yığın kullanınca problem çözüldü: 3'ten büyük bir şey gelince 3'ün üzerine yerleşir, o giderse yine 3 kalır.

> **Ders sonrası not:** Derste max yığınına yalnızca "büyük" değerler kondu. Aynı en büyük değer iki kez push edilirse (örneğin 3, 9, 9), ikinci 9 max yığınına girmez; ilk pop ile 9 max yığınından çıkar ve yığında hâlâ bir 9 varken en büyük 3 görünür. Bu yüzden push'taki koşul "büyük veya eşit" (`>=`) olmalıdır. Python ile denetlendi: `>`

ile 3, 9, 9 dizisinden bir pop sonrası max yığını 3 gösterirken, >= ile doğru olarak 9 gösterir.

Ek bilgi (derste anlatılmadı): Bu çözümde push, pop ve en büyüğü öğrenme işlemlerinin her biri sabit sayıda yığın işlemi yapar, yani $O(1)$'dir. Bedeli, en kötü durumda (değerler artan sırayla gelirse) asıl yığın kadar büyüyebilen ikinci bir yığındır.

Örnek Sorular

Bu derste ekranda ayrı bir soru ya da quiz gösterilmedi; hazır kare ve video da yoktu. Aşağıdaki sorular hocanın derste sözlü olarak öğrencilere bıraktığı ya da sorduğu sorulardır.

Soru 1: peek Fonksiyonu

(derste sözlü, 00:33:45)

Dizi ile gerçekleştirilen yığın için peek fonksiyonunu yazın. Peek pop'a benzer, ama `top`'u azaltmaz; yalnızca yığının en üstündeki, yani `top - 1` numaralı gözdeki değeri verir. Dönüş değeri pop'taki gibi başarıyı gösterir.

Çözüm (derste verilmedi): Yığın boşsa 0 döndürülür; değilse en üstteki değer `value`'ya yazılır ve 1 döndürülür. `top` değişmez.

```
int peek(int *value, stack *s)
{
    if (isEmpty(s))
        return 0;
    *value = s->items[s->top - 1];
    return 1;
}
```

Bu fonksiyon, Bölüm 3'teki yığın koduyla birlikte `gcc -Wall -Wextra` ile uyarısız derlendi. A, B, C push edildikten sonra peek C'yi verdi ve `top` 3'te kaldı; boş yığında peek 0 döndürdü.

Soru 2: Dairesel Kuyrukta dequeue

(derste sözlü, 01:38:19)

Bölüm 7'deki dairesel kuyruk için dequeue fonksiyonunu internete bakmadan yazın. İpucu: pop'taki gibi değer parametreyle dönsün, dönüş değeri başarıyı göstersin; `front` gözündeki değer alınır ve `front` ilerletilir.

Çözüm (derste verilmedi): Kuyruk boşsa 0 döndürülür. Değilse `front` gözündeki değer alınır. Çıkan eleman kuyruktaki son elemansa (`front == rear`), kuyruk boşalmıştır; isEmpty'nin `front == -1` kontrolüyle tutarlı olması için `front` ve `rear` yeniden -1 yapılır. Aksi halde `front`, enqueue'daki `rear` gibi mod ile ilerletilir: `front = (front + 1) % MAX`. Böylece dizinin sonundan başa dönülür.

```
int dequeue(queue *q, int *value)
{
    if (isEmpty(q)) {
        printf("queue is empty");
        return 0;
    }
    *value = q->items[q->front];
    if (q->front == q->rear) {       /* son eleman da çıktı */
        q->front = -1;
        q->rear = -1;
    }
    else
        q->front = (q->front + 1) % MAX;
    return 1;
}
```

Kod, derste yazılan initialize, isEmpty, isFull ve enqueue ile birlikte `MAX = 4` alınarak `gcc -Wall -Wextra` ile uyarısız derlendi ve çalıştırıldı. 10, 20, 30, 40 eklendi; beşinci ekleme "queue is full" ile reddedildi. İki dequeue 10 ve 20'yi verdi (`front = 2`, `rear = 3`). Ardından 50 ve 60 eklendi; `rear` önce 0'a, sonra 1'e döndü ve kuyruk `front == rear + 1` koşuluyla dolu göründü. Boşalana kadar çıkarılınca sıra 30, 40, 50, 60 oldu; sonunda `front` ve `rear` -1'e döndü ve boş kuyruktan dequeue 0 döndürdü.

Soru 3: Bağlı Liste Kuyruğu için isEmpty

(derste sözlü, 02:16:36)

Bağlı liste ile gerçekleştirilen kuyruk için isEmpty fonksiyonunu ayrıca yazın.

> **Çözüm (derste verilmedi):** Kuyruğun boş olduğunu `front`'un `NULL` olması gösterir. Dequeue, son eleman çıkınca `rear`'ı da `NULL` yaptığı için yalnızca `front`'a bakmak yeterlidir.

```
int isEmpty(queue *q)
{
    if (q->front == NULL)
        return 1;
    else
        return 0;
}
```

Bu fonksiyon Bölüm 8'deki kodla birlikte derlendi; dequeue içindeki `front == NULL` kontrolü yerine `isEmpty(q)` kullanıldı ve aynı sonuçlar elde edildi.

Soru 4: Yığındaki En Büyük Eleman

(derste sözlü, 02:20:04)

Yalnızca push, pop ve peek işlemleri yapılabilen bir yığında, herhangi bir anda yığındaki en büyük elemanı nasıl bilirsiniz? Örneğin 3, 1, 7, 4, 9 push edildikten sonra 9 pop edilirse en büyük 7 olmalıdır.

Çözüm (derste verildi): Bir öğrencinin önerisi ve hocanın anlatımıyla, en büyük değerleri sırayla saklayan ikinci bir yığın (max yığını) kullanılır. Push edilen değer o anki en büyükten büyükse (eşit değerler için büyük veya eşitse, bkz. Bölüm 9.3'teki ders sonrası not) max yığınına da push edilir. Pop edilen değer max yığınının üstündekine eşitse oradan da pop edilir. En büyük, her zaman max yığınının üstündedir. Adım adım iz Bölüm 9.3'teki tablodadır.

Çözüm, `>=` koşuluyla C'de yazılıp `gcc -Wall -Wextra` ile derlendi ve çalıştırıldı. 3, 1, 7, 5, 9, 11, 2 push edilince en büyük sırayla 3, 3, 7, 7, 9, 11, 11 oldu; 2, 11, 9, 5, 7 pop edilince sırayla 11, 9, 7, 7, 3 oldu. Bu, derstekiyle aynıdır.

Dersin Toparlaması

Bu derste, elemanlara erişim şekli diziden ve bağlı listeden farklı olan iki veri yapısı görüldü. Yığın son giren ilk çıkar (LIFO) mantığıyla çalışır; undo ve redo, tarayıcıda geri gitme ve fonksiyon çağırma bunun örnekleridir. Yığın bir dizi ve bir `top` indisiyle gerçekleştirildi; push ve pop'un yanında isFull, isEmpty ve peek fonksiyonları yazıldı. Yığının en önemli uygulaması olarak aritmetik ifade değerlendirme incelendi: Infix bir ifade, işlenenler ve işlemciler için iki yığınla tek seferde çözülebilir; ya da önce işlemci yığınıyla postfix'e çevrilip sonra işlenen yığınıyla çözülebilir. İki yolda da `3 * (5 + 8 / 2) - 4` ifadesinin sonucu 23, postfix hali `3 5 8 2 / + * 4 -` oldu.

İkinci yarıda ilk giren ilk çıkar (FIFO) mantığıyla çalışan kuyruk ele alındı. Basit kuyrukta dequeue, elemanları kaydırdığı için $O(n)$'dir. Dairesel kuyruk `front` ve `rear` indislerini mod ile döndürerek, bağlı liste ile kuyruk da yalnızca `front` ve `rear` pointer'larını değiştirerek ekleme ve çıkarmayı $O(1)$ yapar. Ders, yığındaki en büyük elemanı her an bilme sorusuyla bitti; çözüm olarak en büyükleri tutan ikinci bir yığın kullanıldı. Hoca bu sorunun bir sonraki derste yeniden konuşulacağını söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.